Pathfinder Software Design Description

6.1 Introduction

This document presents the architecture and detailed design for the "Pathfinder" project. Pathfinder is a web application designed to assist users, including students, recent graduates, and professionals, in career exploration. It offers career guidance, job recommendations, resume analysis, and real-time support via an AI-powered chatbot. The platform integrates OpenAI for natural language processing, Firebase for data management and authentication, and AWS for hosting.

6.1.1 System Objectives

The primary goals of Pathfinder include:

1. Providing tailored career guidance based on user inputs and preferences.
2. Offering real-time job recommendations and updates.
3. Delivering interactive resume feedback with improvement suggestions.
4. Enabling users to access an AI chatbot for career-related queries.
5. Ensuring secure user data handling and reliable authentication.

6.1.2 Hardware, Software, and Human Interfaces

Hardware Interfaces:

1. **User Devices**: Accessible on desktops, laptops, and mobile devices (iOS/Android).
2. **Cloud Infrastructure**: AWS for backend processing, application hosting, and load balancing.

Software Interfaces:

1. **Frontend**: React.js for a responsive and interactive user interface.
2. **Backend**: Node.js for business logic and Firebase for data storage/authentication.
3. **AI Integration**: OpenAI's API for chatbot interactions.
4. **External APIs**: For retrieving job listings.

Human Interfaces:

1. **User Interface**: A GUI accessible via web browsers (Chrome, Firefox, Safari).
2. **Dashboard Layout**: React-based modules for job search, resume analysis, and chatbot interaction.

6.2 Architectural Design

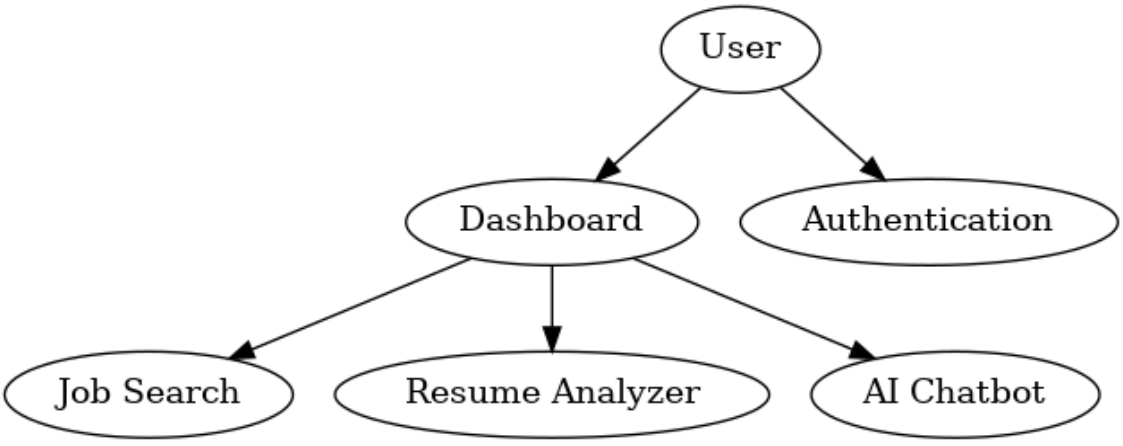
6.2.1 Major Software Components

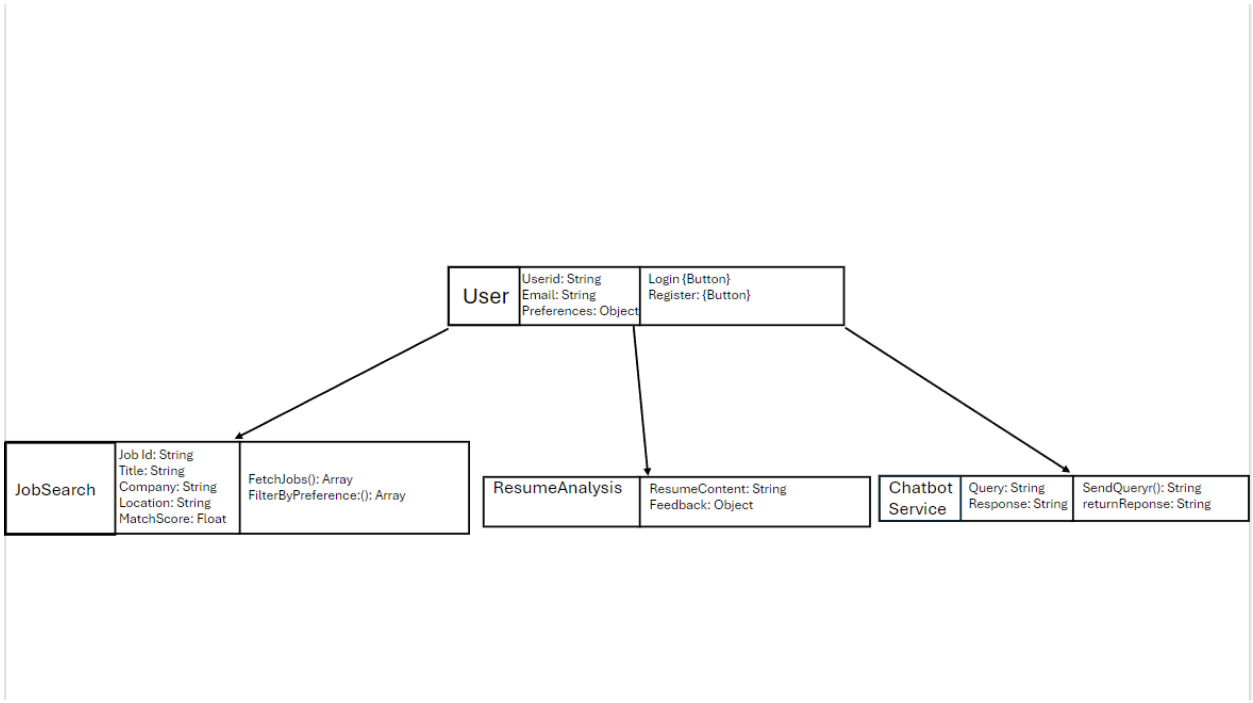
1. **User Authentication Module**: Secure login and registration via Firebase Authentication.
2. **Dashboard Module**: Central user interface for accessing features like career planning, resume analysis, and job matching.
3. **AI Chatbot Module**: Leverages OpenAI's API for natural language responses.
4. **Job Recommendation Module**: Interfaces with job listing APIs to fetch relevant opportunities.
5. **Data Management Module**: Uses Firebase for storing user data, supplemented by AWS for scalability.

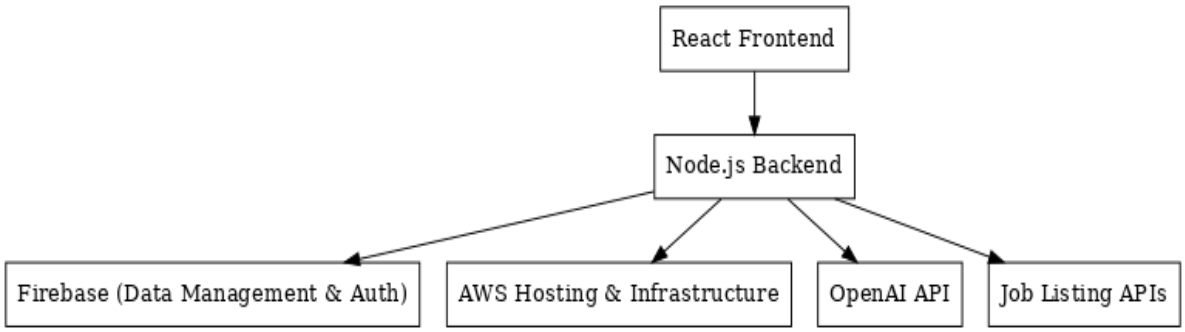
6.2.2 Major Software Interactions

1. **Frontend-Backend Communication**: Facilitated by RESTful HTTP APIs for seamless data exchange.
2. **Backend-Firebase Interaction**: Manages user data and authentication through real-time secure operations.
3. **Backend-OpenAI Integration**: Routes user chatbot queries to OpenAI's API for processing.
4. **Backend-Job API Integration**: Retrieves job listings based on user preferences and profile.

6.2.3 Architectural Design Diagrams







6.3 CSC and CSU Descriptions

6.3.1 Class Descriptions

The following sections detail each class used in the Pathfinder application.

6.3.1.1 User

- **Fields**:
    - `userId`: Unique identifier (String).
    - `email`: User's email (String).
    - `preferences`: User-selected options (Object).

- **Methods**:
  - `login()`: Authenticates user.
  - `register()`: Registers a new user.
  - `updatePreferences()`: Updates user preferences.

6.3.1.2 JobSearch

- **Fields**:
  - `jobId`: Unique identifier (String).
  - `title`: Job title (String).
  - `company`: Employer name (String).
  - `location`: Job location (String).
  - `matchScore`: Relevance score (Float).
- **Methods**:
  - `fetchJobs()`: Retrieves job listings.
  - `filterByPreferences()`: Filters jobs based on user preferences.

6.3.1.3 ResumeAnalyzer

- **Fields**:
  - `resumeContent`: Raw text of user's resume (String).
  - `feedback`: Analysis and suggestions (Object).
- **Methods**:
  - `analyzeResume()`: Analyzes resume structure.
  - `generateSuggestions()`: Produces improvement suggestions.

6.3.1.4 ChatbotService

- **Fields**:
  - `query`: User input query (String).
  - `response`: AI-generated response (String).
- **Methods**:
  - `sendQueryToAI()`: Connects to OpenAI API.
  - `formatResponse()`: Formats the AI's response.

6.3.2 Detailed Interface Descriptions

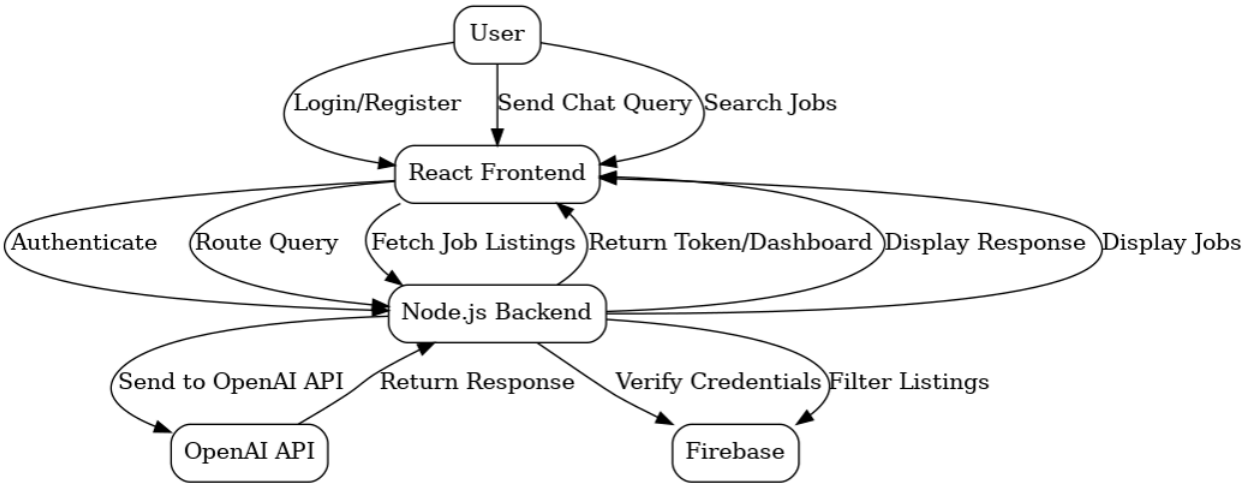
1. **User to Dashboard**:
   - **Input**: User credentials.
   - **Output**: Authentication token and dashboard access.
   - **Interaction**: React frontend communicates with Firebase via backend.
2. **Chatbot API**:
   - **Input**: User queries.

    - **Output**: Chatbot responses.
    - **Interaction**: Backend routes query to OpenAI API.
3. **Job Search API**:
    - **Input**: User preferences.
    - **Output**: Array of job listings.
    - **Interaction**: Backend interacts with external APIs.

6.3.3 Detailed Data Structure Descriptions

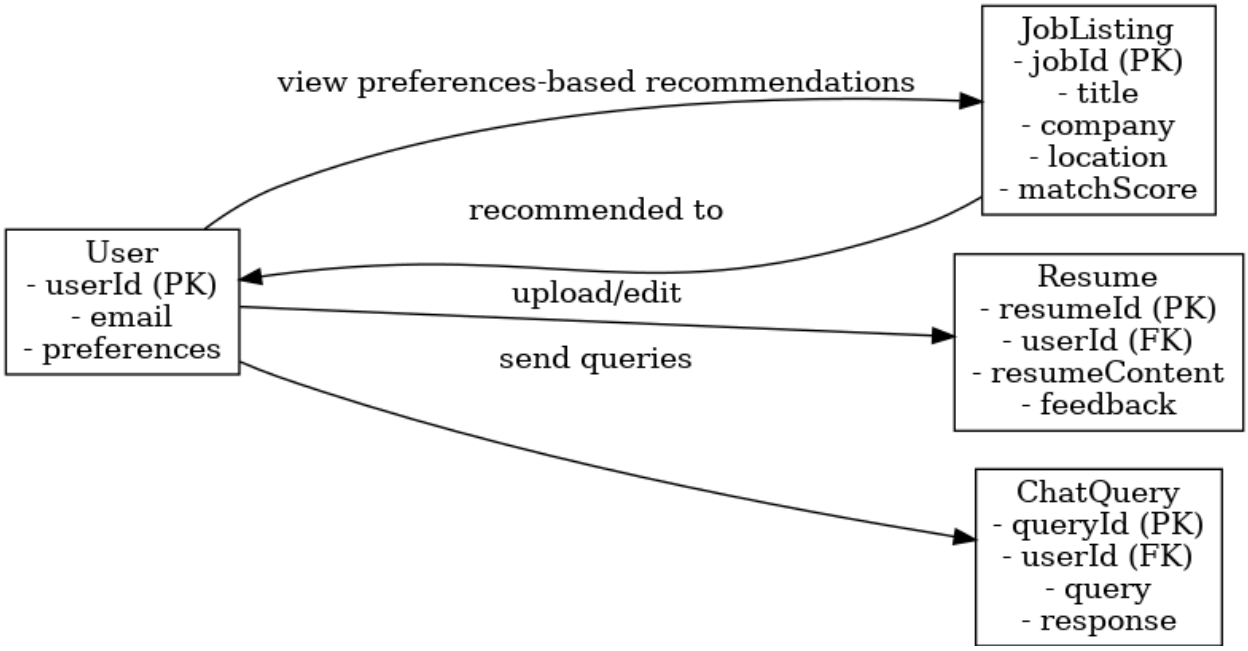
1. **User Data Structure**:
    - `userId`: String, unique identifier.
    - `email`: String, user email.
    - `preferences`: Object, user-selected settings.
2. **Job Listing Data Structure**:
    - `jobId`: String, unique identifier.
    - `title`: String, job title.
    - `company`: String, employer name.
    - `location`: String, job location.
    - `matchScore`: Float, relevance score.

6.3.4 Detailed Design Diagrams



6.4 Database Design and Description

6.4.1 Database Design ER Diagram



6.4.2 Database Access

The database is accessed via Firebase APIs for real-time data storage and retrieval. All database interactions are routed through the backend, ensuring secure and efficient communication.

6.4.3 Database Security

1. **Encryption**: Data is encrypted at rest and during transmission using HTTPS and Firebase's encryption protocols.
2. **Authentication**: Secure authentication protocols through Firebase Authentication.
3. **Access Control**: Role-based access policies to limit data exposure.